
title: "Embedded Security: SSH Tunneling for Modbus/TCP"
author: "Student"

Slide 1: Title

Embedded Security – MASTER Applied Computer Science

Submodule of Selected Topics of Embedded Software Development I

Topic: SSH Tunneling for Modbus/TCP protocol

Slide 2: Agenda

1. What is Modbus/TCP?
 2. Security Vulnerabilities in Modbus/TCP
 3. Securing Communication with SSH Tunneling
 4. Project Architecture & Setup
 5. Traffic Evaluation (Wireshark)
 6. Feasibility on Embedded Systems
 7. Conclusion
-

Slide 3: What is Modbus/TCP?

- **Industry Standard:** Widely used communication protocol in industrial automation (SCADA, PLCs).
 - **Architecture:** Client/Server (formerly Master/Slave) model.
 - **Transport:** Wraps traditional serial Modbus data inside TCP/IP packets.
 - **Default Port:** 502 (We use 5020 for local testing).
 - **Simplicity:** Known for being lightweight and easy to implement.
-

Slide 4: Security Vulnerabilities

Despite its popularity, Modbus/TCP was designed without security in mind.

- **No Encryption:** All data (commands, sensor readings) is sent in plaintext.
 - **No Authentication:** Servers do not verify the identity of the client sending commands.
 - **No Integrity Checks:** No protection against man-in-the-middle data tampering.
 - **Vulnerable to:** Eavesdropping, spoofing, and replay attacks.
-

Slide 5: The Solution: SSH Tunneling

- **Concept:** Wrapping insecure Modbus/TCP traffic inside a secure, encrypted SSH connection.
- **Mechanism:** Port Forwarding.
- **Benefit:** Provides Encryption, Authentication, and Integrity without modifying the legacy Modbus

protocol or the end-devices' Modbus software.

Slide 6: Project Architecture & Setup

Our Test Environment:

1. **Virtual PLC (Server):** Python script (modbus_server.py) using pymodbus, listening on port 5020.
 2. **HMI/Control (Client):** Python script (modbus_client.py) reading/writing registers.
 3. **Network:** Loopback interface (127.0.0.1) for easy traffic monitoring.
-

Slide 7: Implementing the SSH Tunnel

Local Port Forwarding Command:

```
ssh -L 2222:localhost:5020 user@<modbus_server_ip>
```

- Client connects to localhost:2222.
 - SSH Client intercepts, encrypts, and sends to SSH Server.
 - SSH Server decrypts and forwards to localhost:5020 (Modbus Server).
-

Slide 8: Evaluation - Without SSH (Plaintext)

- **Tool:** Wireshark filtering for tcp.port == 5020.
 - **Observation:** Traffic is easily identified as Modbus/TCP.
 - **Risk:** We can read the exact Register Addresses and Values being written. An attacker can completely monitor the industrial process.
-

Slide 9: Evaluation - With SSH (Encrypted)

- **Tool:** Wireshark filtering for SSH traffic (tcp.port == 22).
 - **Observation:** Traffic is identified as SSHv2.
 - **Security Achieved:** The payload is encrypted gibberish. The attacker cannot see the Modbus function codes, nor the register values. The industrial process is hidden.
-

Slide 10: Feasibility on Embedded Systems

Can we put SSH directly on an embedded sensor or PLC?

Challenges:

1. **CPU Overhead:** Cryptographic algorithms (AES, RSA) require significant processing power.
2. **Memory Constraints:** SSH daemons require RAM and Flash storage often unavailable on constrained microcontrollers.
3. **Latency penalty:** Encryption adds jitter, which can disrupt real-time control loops.

- *Note: We created a `modbus_benchmark.py` script to measure this exact latency and CPU overhead by performing 1,000 read requests! It generates a visual bar chart comparing the two scenarios.*
-

Slide 11: Feasibility Conclusion

- **Deeply Embedded Systems (e.g., Cortex-M):** Generally **NOT feasible** due to severe resource limitations.
 - **Edge Gateways / Modern Industrial PCs:** Highly feasible. These devices act as a secure proxy, handling the heavy lifting of SSH while talking plaintext Modbus to local legacy devices.
-

Slide 12: Summary

- Modbus/TCP is critical but insecure.
 - SSH Tunneling successfully mitigates eavesdropping and tampering.
 - Demonstrated via Python implementation and Wireshark capture.
 - Deployment strategy must consider the resource constraints of embedded hardware.
-

Slide 13: Questions?

Thank you for your attention.